

SkyRecon Platform - Detailed Project Documentation

1. Project Overview

SkyRecon is an advanced AI-powered drone intelligence and disaster monitoring platform. It integrates a responsive frontend built with React, Vite, and Tailwind CSS, and a robust backend powered by FastAPI and PostgreSQL. It features real-time mapping, video analysis, telemetry tracking, and comprehensive reporting capabilities.

2. Comprehensive Directory Structure

```
SkyRecon/
├── README.md                # High-level project documentation
├── package.json             # Frontend dependencies and scripts
├── vite.config.js           # Vite bundler configuration
├── tailwind.config.js       # Tailwind CSS configuration
├── index.html               # Entry HTML file
├── src/                     # Frontend source code
│   ├── main.jsx             # React application entry point
│   ├── App.jsx              # Main application component and routing
│   ├── index.css             # Global styles and Tailwind imports
│   ├── assets/               # Static assets (images, icons)
│   ├── components/           # Reusable React components
│   │   ├── Sidebar.jsx       # Navigation sidebar
│   │   ├── TopNav.jsx        # Top navigation bar
│   │   └── ui/                # Granular UI components
│   │       ├── AnimatedCounter.jsx # Animated number counter
│   │       ├── ConfidenceBar.jsx  # AI confidence level visualizer
│   │       ├── DetectionCard.jsx  # Card for AI detections
│   │       ├── FileDropzone.jsx   # Drag-and-drop file upload zone
│   │       ├── GlassCard.jsx      # Glassmorphism styled card contain
│   │       ├── MapWidget.jsx      # Mini map widget component
│   │       ├── SeverityBadge.jsx  # Badge indicating severity levels
│   │       ├── StatCard.jsx       # Statistics display card
│   │       ├── VideoPlayer.jsx    # Video stream/playback component
│   │       └── WeatherWidget.jsx  # Real-time weather display
│   ├── pages/                # Page-level components (Routes)
│   │   ├── LandingPage.jsx      # Introduction and features overview
│   │   ├── Dashboard.jsx        # Main dashboard with statistics
│   │   ├── MapPage.jsx          # Full-screen live tracking map
│   │   ├── MappingPage.jsx      # Drone path and mapping configurations
│   │   ├── DisasterPage.jsx     # Disaster event monitoring
│   │   ├── ReportsPage.jsx      # Report generation and viewing
│   │   └── AdminPage.jsx        # System administration and settings
├── backend/                 # Backend source code
│   ├── requirements.txt       # Python dependencies
│   ├── app/                  # Main FastAPI application
│   │   └── main.py            # FastAPI application initialization
```

			database.py	# SQLAlchemy database connection setup
			models/	# SQLAlchemy ORM models
			models.py	# Database schema definitions
			api/	# API router definitions
			routes.py	# Endpoint definitions (e.g., /api/missio
			core/	# Core configurations and utilities
			config.py	# Environment variable configuration
			security.py	# Authentication and authorization logic
			seeder.py	# Database seeding scripts for mock data
			sql/	# Raw SQL scripts
			procedures.sql	# PL/pgSQL stored procedures for complex

3. Frontend Files: Functions, Modules, and APIs

Location: `src/` directory. **Core Technologies:** React 19, React Router DOM, Tailwind CSS 4, Framer Motion, Recharts, React-Leaflet.

3.1. Page Components (`src/pages/`)

- **LandingPage.jsx:** Serves as the entry point for users, highlighting platform capabilities using `framer-motion` for animations and `lucide-react` for icons.
- **Dashboard.jsx:** Provides a high-level overview of system status, active drones, recent alerts, and quick stats utilizing `recharts` for data visualization.
- **MapPage.jsx:** Displays live drone locations, paths, and points of interest. See Section 7 for Map details.
- **DisasterPage.jsx:** Dedicated interface for monitoring disaster zones, AI damage assessments, and emergency resource allocation. See Section 8 for AI & Video Analysis details.
- **ReportsPage.jsx:** Allows users to generate, view, and export detailed mission and analytical reports using `FileDropzone`.
- **AdminPage.jsx:** Management interface for users, drones, and system settings.

3.2. UI Components (`src/components/ui/`)

- **GlassCard.jsx:** A styled wrapper component utilizing Tailwind CSS for glassmorphism effects (`backdrop-blur`, translucent backgrounds).
- **AnimatedCounter.jsx:** Uses `framer-motion` to smoothly animate numerical changes (e.g., increasing drone count).
- **DetectionCard.jsx:** Displays AI analysis results (e.g., "Fire Detected", "Confidence: 95%").
- **FileDropzone.jsx:** Utilizes `react-dropzone` or native HTML5 Drag-and-Drop APIs for file uploads.

4. Backend Files: Functions, Modules, and APIs

Location: `backend/` directory. **Core Technologies:** Python 3.x, FastAPI, SQLAlchemy, PostgreSQL, Pydantic.

4.1. Core Application (`backend/app/`)

- **main.py:** The entry point for the FastAPI server. Configures CORS (Cross-Origin

Resource Sharing) middlewares and registers API routers. Uses `fastapi.FastAPI` and `fastapi.middleware.cors.CORSMiddleware`.

- **database.py:** Establishes the connection to the PostgreSQL database using SQLAlchemy. Uses `sqlalchemy.create_engine` and `sqlalchemy.orm.sessionmaker`.
- **models/models.py:** Defines the database tables as SQLAlchemy ORM classes (e.g., `User`, `Drone`, `Mission`, `Telemetry`, `Alert`).
- **core/seeder.py:** Contains logic to populate the database with initial/mock data for testing and development.

4.2. Database Logic (`backend/sql/procedures.sql`)

- **Function:** Contains 12 PL/pgSQL stored procedures designed to offload complex, data-heavy operations from the FastAPI backend directly to the PostgreSQL database.
- **Key Procedures:**
 - `calculate_mission_stats(mission_id)`: Aggregates telemetry data to calculate distance, duration, and average speed.
 - `cascade_delete_drone(drone_id)`: Safely removes a drone and all its associated telemetry, missions, and alerts to maintain referential integrity.
 - `generate_disaster_report(event_id)`: Compiles data from various tables to produce a comprehensive summary of a disaster event.

5. API Endpoints Overview (FastAPI)

- **Authentication:** `/api/auth/login`, `/api/auth/register` (Uses JWT tokens for security).
- **Drones:** `/api/drones` (GET list, POST create, PUT update status).
- **Missions:** `/api/missions/{id}` (Retrieve mission details).
- **GIS Markers:** `/api/v1/dashboard/map-markers` (Fetches active database markers for the map).

6. Map Technology & Implementation

The mapping module (`src/pages/MapPage.jsx`) is a critical component of SkyRecon, utilizing advanced GIS (Geographic Information System) logic.

- **Core Libraries:** Uses `leaflet` and `@react-leaflet/core` along with `react-leaflet`.
- **Map Features:**
 - **Tile Layers:** Supports dynamic switching between three map views:
 1. **Dark Mode:** Rendered via CartoCDN (`https://{s}.basemaps.cartocdn.com/dark_all...`)
 2. **Satellite View:** High-fidelity imagery via ArcGIS (`https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery...`)
 3. **Street View:** OpenStreetMap tiles.
 - **Geolocation (`navigator.geolocation`):** Auto-detects user location on mount and provides a "Locate Me" fly-to functionality using Leaflet's `map.flyTo` method.

- **Data Visualization:** Implements Heatmap overlays representing detection density (rendered using `react-leaflet`'s `Circle` component with dynamic radii).
- **Custom Markers:** Uses `L.DivIcon` to render pulsating, color-coded HTML markers that denote severity levels across different regions of India (e.g., Red for Level 5 Flood Zones, Orange for Traffic Corridors).

7. AI Features & Video Analysis Engine

The AI and disaster detection capabilities are housed within `src/pages/DisasterPage.jsx`, acting as the "Disaster Assessment & Rescue Workspace".

- **AI Inference Models:**

- The platform leverages **YOLO-World** (mocked in the UI) for standalone edge inference, capable of functioning in low-bandwidth (Offline) environments without centralized cloud processing.
- Provides high-priority disaster scans that identify hazards, rank structural distress (Severity Levels 1-5), and calculate required emergency resources dynamically.

- **Video Processing Workflow:**

1. **Ingestion:** Users upload drone surveillance footage via `FileDropzone.jsx`.
2. **Analysis Simulation:** Video is played using native HTML5 `<video>`, while `framer-motion` and `lucide-react` simulate buffer frame scans and inference sweeps across the footage.
3. **Result Generation:** Generates comprehensive detection logs including timestamps, classification (e.g., "Flooded Residential Area"), and severity.

- **Dynamic Resource Estimator:** An algorithmic function (`calculateResources`) that automatically computes the required Rescue Teams, Ambulances, Rescue Boats, and Support Staff based on the exact severity multipliers of the detected incidents.
- **Audio Announcements:** Utilizes native browser API `window.speechSynthesis` to broadcast vocal alarms and emergency evacuation notices when Critical Level 5 threats are identified by the AI.
- **Reporting:** Automatically formats the AI analysis data into a downloadable text/blob file mimicking DOCX/PDF export of the Disaster Assessment.